Q1. Discuss the concept of Excel Styles, illustrating your answer with an example, and explain how they contribute to effective and consistent spreadsheet formatting.

Concept of Excel Styles :

Excel Styles are pre-defined formatting rules that can be applied to cells, ranges, tables or the entire workbook to quickly change their appearance. A style is a combination of multiple formatting attributes such as font, font size, color, number format, alignment, borders, fill color, etc. Using styles helps in maintaining uniformity and saving time.

Example :

Consider the following sales data.

Product	Jan	Feb	Mar	Total
Laptop	45000	52000	48000	145000
Mobile	30000	28000	35000	93000
Tablet	15000	18000	20000	53000
Total	90000	98000	103000	290000

We can apply built-in or custom styles as shown below.

Styles Gallery (Home → Styles Group)

Normal	Bad	Good	Neutral	Calculation
Check Cell	Heading 1	Heading 2	Title	Total

Applying Styles :

- Select the data range.
- Go to Home tab → Styles group.
- Choose a suitable style (e.g., Heading 1 for title, Total for total row).
- The selected style is applied instantly with consistent formatting.

How Excel Styles contribute to effective and consistent formatting :

1. Consistency : Ensures uniform look and feel across the entire workbook.
2. Time Saving : Apply a style once and reuse it anywhere.
3. Easy Maintenance : Change the style definition and the formatting updates everywhere it is used.
4. Professional Appearance : Gives a clean, well structured and professional look to spreadsheets.
5. Scalability : Useful for large datasets and team based work.

Conclusion : Excel Styles are a powerful feature that promotes consistency, saves time and improves the readability and professionalism of spreadsheets.

Q2. Demonstrate with examples how to create, apply, and modify cell styles to enhance the readability and consistency of large data sets.

Excel cell styles help us format cells consistently. We can create custom styles, apply them quickly and modify them whenever needed.

1. Creating Cell Styles (Example)

- Home tab → Styles group → Cell Styles → New Cell Style.
- Give a name, choose a style type (e.g., Number),
- Click Format to set Font, Border, Fill, Number format, Alignment.
- Click OK.

Example: Create the following styles for a sales report.

Style Name	Purpose	Key Formatting
Header	For table headings	Bold, White font, Dark Blue fill, Center align
Currency	For amount values	₹ Currency format, 2 decimals, Right align
Good	For good performance	Green fill, Dark Green font
Bad	For poor performance	Light Red fill, Dark Red font
Total	For totals	Bold, Light Gray fill, Top border thick

Steps:
1. Home → Styles → Cell Styles → New Cell Style
2. Name the style
3. Click Format and set options
4. OK

2. Applying Cell Styles (Example)

- Select the cells or range → Home tab → Cell Styles → choose the style.

Example: Sales Data

	A	B	C	D	E
	Product	Jan Sales	Feb Sales	Mar Sales	Total
2	Laptop	45000	52000	48000	145000
3	Mobile	30000	28000	35000	93000
4	Tablet	15000	18000	20000	53000
5	**Total**	**90000**	**98000**	**103000**	**290000**

Applied Styles:
- Row 1 → Header
- B2:E4 → Currency
- Row 5 → Total

3. Modifying Cell Styles (Example)

- Home tab → Cell Styles → right click the style → Modify.
- Change the formatting (e.g., font color, fill, border).
- Click OK. All cells using this style will update automatically.

Example: Modify "Good" style

Before Modification

Product	Performance
Laptop	Good
Mobile	Good
Tablet	Bad

Modify "Good" style → change fill to light blue and font to dark blue →

After Modification (auto updated)

Product	Performance
Laptop	Good
Mobile	Good
Tablet	Bad

4. Benefits for Large Data Sets

- Consistency : Same look and feel across thousands of cells.
- Readability : Important data stands out (e.g., headers, totals, conditions).
- Time Saving : Apply formats in one click.
- Easy Maintenance : Modify once, updates everywhere.
- Professional Look : Clean, well-organized, easy to understand reports.

Q3. Explain how pivot charts complement pivot tables with visual representation of data.

Pivot Charts are the graphical (visual) representation of the summarized data shown in pivot tables. While pivot tables present data in rows and columns, pivot charts convert the same data into charts / graphs like column, bar, pie, line, etc., which make trends, patterns, comparisons and outliers easy to understand.

How Pivot Charts complement Pivot Tables :

1. **Visual Impact** : Charts give a quick visual summary of complex numbers.
2. **Trend Identification** : Easier to see trends over time, highs / lows, and patterns.
3. **Comparison** : Charts make comparison between categories more effective.
4. **Better Decision Making** : Visuals are understood faster and help in data-driven decisions.
5. **Interactivity** : Both pivot tables and pivot charts are interactive. Changing filters or slicers updates both.

Example : Sales Data

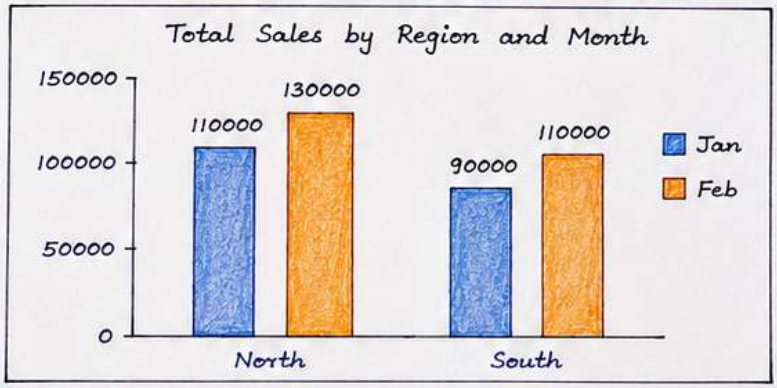
Region	Product	Month	Sales (₹)
North	Laptop	Jan	65000
North	Laptop	Feb	75000
North	Mobile	Jan	45000
North	Mobile	Feb	55000
South	Laptop	Jan	50000
South	Laptop	Feb	60000
South	Mobile	Jan	40000
South	Mobile	Feb	50000

Create Pivot Table →

Pivot Table (Summary)

Sum of Sales (₹)	Column Labels		
Row Labels	Jan	Feb	Grand Total
North	110000	130000	240000
South	90000	110000	200000
Grand Total	200000	240000	440000

Pivot Chart (Visual Representation of the Above Pivot Table)



How the Chart Complements the Table :

- The chart quickly shows that North has higher sales than South.
- It is easy to see that Feb sales are higher than Jan in both regions.
- Comparison and trend are clear at a glance.
- It helps identify which region or month performed better.

Conclusion :

Pivot charts complement pivot tables by transforming summarized data into meaningful visuals, making reports more effective, easy to understand and impactful for analysis and decision making.

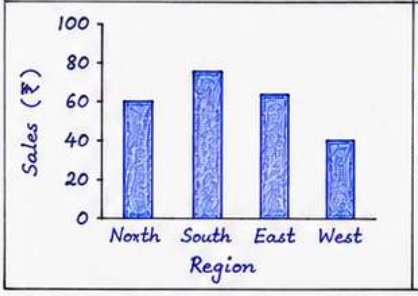
Q4. Compare and contrast different chart types (Column, Line, Pie, and Bar). Discuss scenarios where each chart type is most effective.

Answer: Excel provides a variety of chart types to visualize data. Each chart type has its own strengths and is suitable for specific kinds of data and analysis.

Feature	Column Chart	Line Chart	Pie Chart	Bar Chart
What it shows	Compares values across categories using vertical bars.	Shows trends and changes over time using lines.	Shows part-to-whole relationship using slices of a circle.	Compares values across categories using horizontal bars.
Best for	Comparing discrete categories.	Showing trends over time or continuous data.	Showing percentage or proportion of a whole (100%).	Comparing categories, especially with long labels or many items.
Data Type	Categorical data with one or more series.	Time series data or ordered data.	One data series (percentages / parts).	Categorical data with one or more series.
Orientation	Vertical	Lines on 2-D plot	Circular	Horizontal
Easy to compare	Yes, across categories.	Yes, over time.	Difficult for many slices.	Yes, across categories (better with long labels).
Best When	Few to moderate categories.	Showing trends, patterns, seasonality, growth.	Few categories (up to 5-7 slices max).	Many categories or long names.
Example Use Cases	Sales by region, Quarterly profit comparison.	Stock prices over time, Monthly sales trend, Temperature over days.	Market share of companies, Budget breakdown.	Product-wise sales, Survey results, Performance by employee.

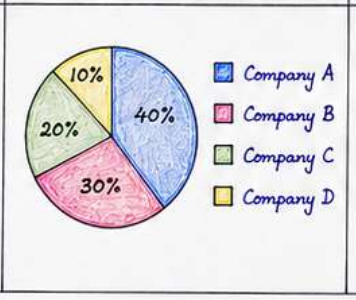
Visual Examples:

Column Chart – Sales by Region

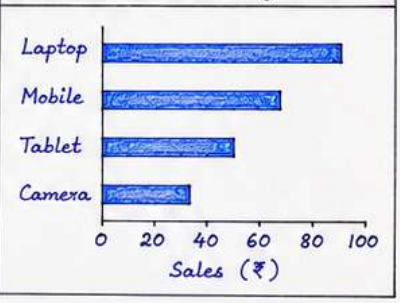


Line Chart – Sales Trend

Pie Chart – Market Share



Bar Chart – Sales by Product



When to Use (Summary):

- Column Chart : When you want to compare values across a set of categories.
- Line Chart : When you want to show how values change over time.
- Pie Chart : When you want to show how parts contribute to a whole.
- Bar Chart : When you want to compare values across many categories or when category names are long and need more space.

Conclusion: Choosing the right chart type helps in effectively communicating insights and making data easy to understand.

Q5. Evaluate how using Slicers improves interactivity and data visualization in PivotTables and PivotCharts. Provide examples.

Slicers are visual filtering controls that allow users to quickly filter data in PivotTables and PivotCharts. They provide buttons for each item in a field, making it easy to filter data with a single click.

How Slicers Improve Interactivity and Data Visualization

1. **Easy and Intuitive Filtering**
   Users can filter data by clicking on slicer buttons instead of using drop-down filter menus.
2. **Enhanced Interactivity**
   Slicers make reports more dynamic. Multiple slicers can be used together to drill down into data.
3. **Better Data Visualization**
   Slicers work seamlessly with PivotCharts, allowing instant visual updates based on selection.
4. **Clearer Insights**
   Helps users focus on relevant data, hiding unwanted information and highlighting trends.
5. **User-Friendly Dashboards**
   Slicers add a professional look to dashboards and make them more user-friendly.
6. **Connected Reports**
   One slicer can control multiple PivotTables and PivotCharts (using Report Connections).

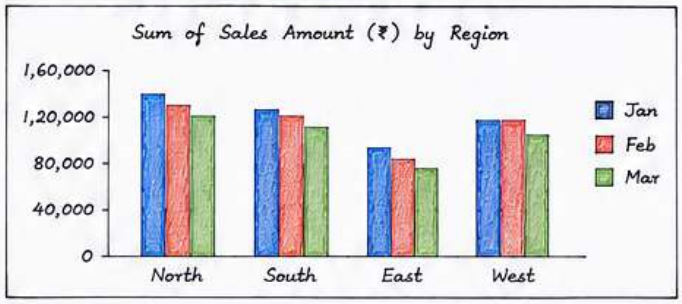
Example: Sales Analysis Report

Dataset contains fields: Date, Region, Product, Salesperson, Sales Amount.

PivotTable (Without Slicer)

Row Labels	Sum of Sales Amount (₹)			
	Jan	Feb	Mar	Grand Total
North	1,25,000	1,40,000	1,35,000	4,00,000
South	1,10,000	1,20,000	1,15,000	3,45,000
East	90,000	1,00,000	95,000	2,85,000
West	1,00,000	1,10,000	1,05,000	3,15,000
Grand Total	4,25,000	4,70,000	4,50,000	13,45,000

PivotChart (Column Chart)



Using Slicers

How it works:

- Click on a slicer item (e.g., North, Laptop, Amit).
- PivotTable and PivotChart update instantly.
- Multiple slicers can be used together for more refined filtering (e.g., North + Laptop + Amit).

Examples of Slicer Usage and Impact

1. Select Region = North

Row Labels	Sum of Sales Amount (₹)			
	Jan	Feb	Mar	Grand Total
North	1,25,000	1,40,000	1,35,000	4,00,000

Impact: Report now shows only North region data. Chart displays sales trend for North only.

Business Insight: North has the highest sales in February. Focus marketing efforts for March in North region.

2. Select Product = Laptop and Region = South

Row Labels	Sum of Sales Amount (₹)			
	Jan	Feb	Mar	Grand Total
South — Laptop	60,000	65,000	62,000	1,87,000

Impact: Report shows Laptop sales in South region only. Helps to analyze product performance in a specific region.

Business Insight: Laptop sales in South are steady. Plan promotions to increase sales in March.

Conclusion

Slicers make PivotTables and PivotCharts highly interactive, visually appealing, and easy to use. They help users explore data quickly, discover trends, and make informed decisions. Slicers turn static reports into dynamic dashboards.

Q6. Demonstrate how to restrict and sort data in SQL using WHERE, ORDER BY, and DISTINCT clauses with appropriate examples.

1. Sample Table

Table : Employees

EmpID	EmpName	Dept	Salary	City	JoinDate
101	Rohan	IT	60000	Pune	2022-01-15
102	Neha	HR	45000	Mumbai	2021-06-20
103	Amit	IT	70000	Pune	2020-03-10
104	Priya	Finance	50000	Delhi	2022-07-01
105	Kiran	IT	65000	Bangalore	2021-11-05
106	Sneha	HR	45000	Pune	2023-02-18
107	Vijay	Finance	55000	Mumbai	2020-09-30

Purpose :

- WHERE clause is used to restrict (filter) rows.
- ORDER BY clause is used to sort the result.
- DISTINCT clause is used to remove duplicate values.

2. Restricting Data using WHERE clause

WHERE clause filters rows based on a condition.

Example 1 : Employees working in IT Department

```
SELECT * FROM Employees
WHERE Dept = 'IT';
```

Result :

EmpID	EmpName	Dept	Salary	City	JoinDate
101	Rohan	IT	60000	Pune	2022-01-15
103	Amit	IT	70000	Pune	2020-03-10
105	Kiran	IT	65000	Bangalore	2021-11-05

Example 2 : Employees with Salary greater than 50000

```
SELECT EmpName, Salary FROM Employees
WHERE Salary > 50000;
```

Result :

EmpName	Salary
Rohan	60000
Amit	70000
Kiran	65000
Vijay	55000

Example 3 : Employees in Pune

```
SELECT EmpName, City FROM Employees
WHERE City = 'Pune';
```

Result :

EmpName	City
Rohan	Pune
Amit	Pune
Sneha	Pune

3. Sorting Data using ORDER BY clause

ORDER BY clause sorts the result in ascending (ASC) or descending (DESC) order.

Example 1 : Sort by Salary in Ascending order

```
SELECT EmpName, Salary FROM Employees
ORDER BY Salary ASC;
```

Result :

EmpName	Salary
Neha	45000
Sneha	45000
Priya	50000
Vijay	55000
Rohan	60000
Kiran	65000
Amit	70000

Example 2 : Sort by Salary in Descending

```
SELECT EmpName, Salary FROM Employees
ORDER BY Salary DESC;
```

Result :

EmpName	Salary
Amit	70000
Kiran	65000
Rohan	60000
Vijay	55000
Priya	50000
Neha	45000
Sneha	45000

Example 3 : Sort by JoinDate (Oldest first)

```
SELECT EmpName, JoinDate FROM Employees
ORDER BY JoinDate ASC;
```

Result :

EmpName	JoinDate
Amit	2020-03-10
Vijay	2020-09-30
Neha	2021-06-20
Kiran	2021-11-05
Rohan	2022-01-15
Priya	2022-07-01
Sneha	2023-02-18

4. Removing Duplicates using DISTINCT clause

DISTINCT clause returns only unique (non-duplicate) values.

Example 1 : Distinct Departments

```
SELECT DISTINCT Dept FROM Employees;
```

Result :

Dept
IT
HR
Finance

Example 2 : Distinct Cities

```
SELECT DISTINCT City FROM Employees;
```

Result :

City
Pune
Mumbai
Delhi
Bangalore

Example 3 : Distinct Salaries

```
SELECT DISTINCT Salary FROM Employees;
```

Result :

Salary
60000
45000
70000
50000
65000
55000

5. Using WHERE, ORDER BY and DISTINCT together

Example : Get distinct departments of employees with salary > 50000, sorted alphabetically.

```
SELECT DISTINCT Dept FROM Employees
WHERE Salary > 50000
ORDER BY Dept ASC;
```

Result :

Dept
Finance
IT

Example : Get employees from Pune, sorted by salary (high to low)

```
SELECT EmpName, Salary FROM Employees
WHERE City = 'Pune'
ORDER BY Salary DESC;
```

Result :

EmpName	Salary
Amit	70000
Rohan	60000
Sneha	45000

Conclusion : WHERE clause restricts rows based on conditions, ORDER BY sorts the result, and DISTINCT removes duplicates. Together, they help in efficient data retrieval and meaningful results.

Q7. Discuss SQL joins for displaying data from multiple tables with an example.

SQL JOIN is used to combine rows from two or more tables based on a related column between them. Joins help in retrieving meaningful information from normalized databases.

Example : Consider the following tables.

Table : Employees (Emp)

EmpID	EmpName	DeptID	Salary
1	Alice	10	60000
2	Bob	20	50000
3	Charlie	10	55000
4	David	30	45000
5	Eva	NULL	40000

Table : Departments (Dept)

DeptID	DeptName	Location
10	HR	Mumbai
20	IT	Pune
30	Finance	Delhi
40	Marketing	Bangalore

The Emp.DeptID is a foreign key referencing Dept.DeptID.

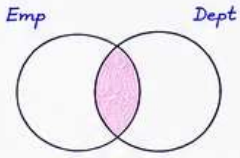
1. INNER JOIN

Returns only matching rows from both tables.

```
SELECT Emp.EmpID, Emp.EmpName, Dept.DeptName, Dept.Location
FROM Employees Emp
 INNER JOIN Departments Dept
    ON Emp.DeptID = Dept.DeptID;
```

Result :

EmpID	EmpName	DeptName	Location
1	Alice	HR	Mumbai
2	Bob	IT	Pune
3	Charlie	HR	Mumbai
4	David	Finance	Delhi



Only Matching Rows

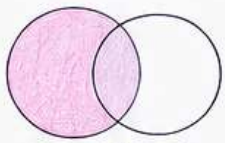
2. LEFT JOIN (LEFT OUTER JOIN)

Returns all rows from left table (Employees) and matching rows from right table.

```
SELECT Emp.EmpID, Emp.EmpName, Dept.DeptName, Dept.Location
FROM Employees Emp
 LEFT JOIN Departments Dept
   ON Emp.DeptID = Dept.DeptID;
```

Result :

EmpID	EmpName	DeptName	Location
1	Alice	HR	Mumbai
2	Bob	IT	Pune
3	Charlie	HR	Mumbai
4	David	Finance	Delhi
5	Eva	NULL	NULL


All Rows from Emp + Matching from Dept*

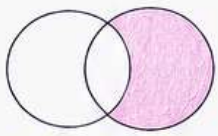
3. RIGHT JOIN (RIGHT OUTER JOIN)

Returns all rows from right table (Departments) and matching rows from left table.

```
SELECT Emp.EmpID, Emp.EmpName, Dept.DeptName, Dept.Location
FROM Employees Emp
 RIGHT JOIN Departments Dept
   ON Emp.DeptID = Dept.DeptID;
```

Result :

EmpID	EmpName	DeptName	Location
1	Alice	HR	Mumbai
2	Bob	IT	Pune
3	Charlie	HR	Mumbai
4	David	Finance	Delhi
NULL	NULL	Marketing	Bangalore


All Rows from Dept + Matching from Emp

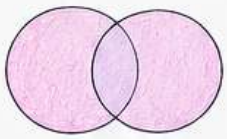
4. FULL OUTER JOIN

Returns all rows when there is a match in either left or right table.

```
SELECT Emp.EmpID, Emp.EmpName, Dept.DeptName, Dept.Location
FROM Employees Emp
 FULL OUTER JOIN Departments Dept
   ON Emp.DeptID = Dept.DeptID;
```

Result :

EmpID	EmpName	DeptName	Location
1	Alice	HR	Mumbai
2	Bob	IT	Pune
3	Charlie	HR	Mumbai
4	David	Finance	Delhi
5	Eva	NULL	NULL
NULL	NULL	Marketing	Bangalore


All Rows from Both Tables

5. CROSS JOIN

Returns Cartesian product (every row of left table with every row of right table).

```
SELECT Emp.EmpID, Emp.EmpName, Dept.DeptName
FROM Employees Emp
 CROSS JOIN Departments Dept;
```

Result (First few rows) :

EmpID	EmpName	DeptName
1	Alice	HR
1	Alice	IT
1	Alice	Finance
...	...	...

Total Rows = 5 x 4 = 20

Use CROSS JOIN carefully as it can produce very large result sets.

Summary of Joins

Join Type	Returns	Includes Unmatched Rows?
INNER JOIN	Only matching rows	No
LEFT JOIN	All rows from left + matching right	Yes (left table)
RIGHT JOIN	All rows from right + matching left	Yes (right table)
FULL OUTER JOIN	All rows from both tables	Yes (both tables)
CROSS JOIN	Cartesian product of both tables	Not applicable

When to use?

- INNER JOIN – When you need only related data.
- LEFT JOIN – When all data from left table is important.
- RIGHT JOIN – When all data from right table is important.
- FULL OUTER JOIN – When you need all data from both tables.
- CROSS JOIN – For generating combinations (rarely used in reporting).

Q8. Explain Data Manipulation Language (DML) statements — INSERT, UPDATE, DELETE, and MERGE. Provide syntax and examples for each.

DML (Data Manipulation Language) is used to manipulate data that is already stored in the database tables.

Sample Table : Emp

EmpID	EmpName	DeptID	Salary	City
101	Alice	10	60000	Pune
102	Bob	20	50000	Mumbai
103	Charlie	10	55000	Pune
104	David	30	45000	Delhi

Another Table for MERGE Example : Emp_Staging

EmpID	EmpName	DeptID	Salary	City
102	Bob	20	52000	Mumbai
105	Eva	30	48000	Pune
103	Charlie	10	56000	Pune

1. INSERT Statement

- Used to insert new rows into a table.

Syntax :

```
INSERT INTO table_name (column1, column2, ...)
VALUES (value1, value2, ...);
```

Example : Insert a new employee.

```
INSERT INTO Emp (EmpID, EmpName, DeptID, Salary, City)
VALUES (105, 'Eva', 30, 48000, 'Pune');
```

Result :

EmpID	EmpName	DeptID	Salary	City
101	Alice	10	60000	Pune
102	Bob	20	50000	Mumbai
103	Charlie	10	55000	Pune
104	David	30	45000	Delhi
105	Eva	30	48000	Pune

2. UPDATE Statement

- Used to modify existing data in a table.

Syntax :

```
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

Example : Give a salary increment to all Pune employees.

```
UPDATE Emp
SET Salary = Salary + 5000
WHERE City = 'Pune';
```

Result :

EmpID	EmpName	DeptID	Salary	City
101	Alice	10	65000	Pune
102	Bob	20	50000	Mumbai
103	Charlie	10	60000	Pune
104	David	30	45000	Delhi
105	Eva	30	53000	Pune

3. DELETE Statement

- Used to remove rows from a table.

Syntax :

```
DELETE FROM table_name
WHERE condition;
```

Example : Delete employee with EmpID = 104.

```
DELETE FROM Emp
WHERE EmpID = 104;
```

Result :

EmpID	EmpName	DeptID	Salary	City
101	Alice	10	65000	Pune
102	Bob	20	50000	Mumbai
103	Charlie	10	60000	Pune
105	Eva	30	53000	Pune

(Row with EmpID = 104 is removed)

4. MERGE Statement

- Used to insert new rows and update existing rows in a single statement by comparing data from a source table with a target table.

Syntax :

```
MERGE INTO target_table AS T
USING source_table AS S
ON (T.key_column = S.key_column)
WHEN MATCHED THEN
    UPDATE SET T.column1 = S.column1, ...
WHEN NOT MATCHED THEN
    INSERT (column1, column2, ...)
    VALUES (S.column1, S.column2, ...);
```

Example : Merge data from Emp_Staging into Emp table.

```
MERGE INTO Emp AS T
USING Emp_Staging AS S
ON (T.EmpID = S.EmpID)
WHEN MATCHED THEN
    UPDATE SET T.Salary = S.Salary, T.City = S.City
WHEN NOT MATCHED THEN
    INSERT (EmpID, EmpName, DeptID, Salary, City)
    VALUES (S.EmpID, S.EmpName, S.DeptID, S.Salary, S.City);
```

Result :

EmpID	EmpName	DeptID	Salary	City	
101	Alice	10	65000	Pune	
102	Bob	20	52000	Mumbai	← Updated
103	Charlie	10	56000	Pune	← Updated
105	Eva	30	48000	Pune	← Inserted

★ Summary of DML Statements

- INSERT → Adds new data into a table.
- UPDATE → Modifies existing data based on a condition.
- DELETE → Removes data from a table based on a condition.
- MERGE → Combines INSERT and UPDATE operations in one statement.

o o o